Chapter 11: Reusable User Embeddings

11.1 Introduction

Chapter 10 established that item embeddings are the foundational representation layer of a recommendation system. A well-trained item embedding space places similar products, articles, or songs near each other, enabling efficient retrieval and meaningful feature inputs for downstream models. But personalization is ultimately about matching users to items. We need a way to represent users in the same (or a compatible) embedding space as items.

A user embedding is a dense vector that summarizes a user's preferences, behavior, and context in a form that can be directly compared against item embeddings. When a user's embedding is close to an item's embedding in **latent** space, the system infers that the user would find that item relevant. This simple geometric relationship that a proximity in embedding space as a proxy for relevance is the core assumption for the candidate retrieval stage in virtually every large-scale recommendation system deployed today.

The fundamental question of this chapter is deceptively simple: given a user's interaction history—a sequence of items they have browsed, clicked, or purchased[1]—how do we construct a single vector that captures their preferences? Formally:

```
user_embedding = f(item_embedding_1, item_embedding_2, ..., item_embedding_N)
```

The question is what f should be. This chapter explores three paradigms for constructing f, each representing a different level of sophistication and a different set of assumptions about what matters in a user's history.

Section 11.2 covers aggregation baselines: simple mathematical operations (mean, weighted mean, exponential decay) over item embeddings that require no training. These methods ask whether the user can be adequately represented as the average of what they consumed. Section 11.3 introduces sequence models (SASRec and GRU4Rec), which are learned neural networks that process the interaction sequence and discover which items in the history are most predictive of the next action. These models ask whether order matters. Section 11.4 presents graph neural networks (LightGCN) which is a graph-based approach that learns user and item embeddings jointly from the user–item interaction graph, capturing collaborative filtering signal through multi-hop neighborhood aggregation. This approach asks whether we can learn from the collective behavior of all users.

In the companion tutorials, we evaluate all approaches on two datasets that represent the two common patterns in production systems: Amazon KDD Cup 2023 (anonymous shopping sessions with short sequences) and MIND (identified news readers with longer

[1] The readers are advised to not take the word 'Purchase' in the literal. Instead this book treats it as a placeholder term for the most consequential step that the user can do in the system. For non-retail use cases, this definition could vary such as shares, reposts, etc.

click histories). This breadth is intended to reveal that the best approach depends critically on the data regime, and that simple methods are surprisingly hard to beat in certain contexts.

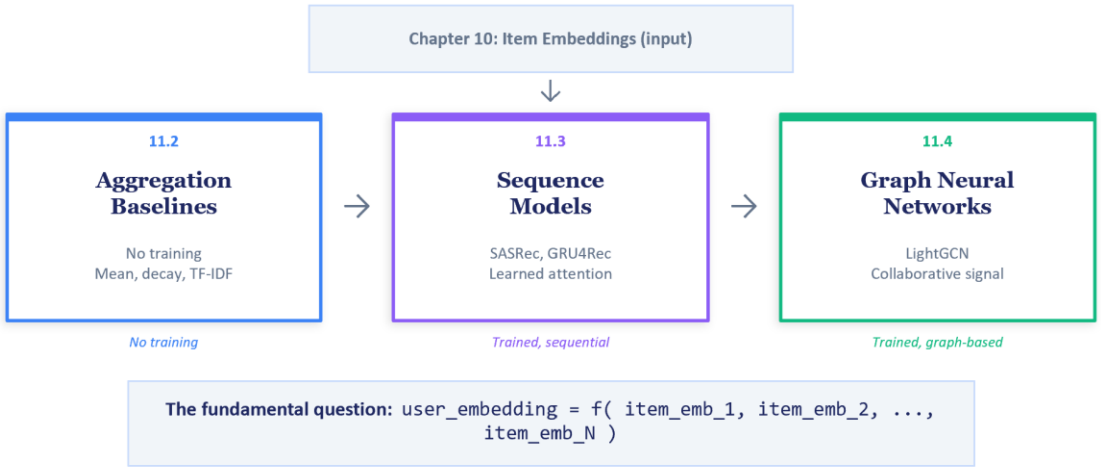


FIGURE 1: Chapter 11 roadmap showing the three paradigm sections (Aggregation, Sequence, Graph) and how they connect to Chapter 10's item embeddings as inputs.

11.2 Aggregation Baselines (No Training Required)

The simplest user embedding is the average of the item embeddings the user has interacted with. This "bag of items" assumption treats a user as the centroid of their interaction history in embedding space. A user who browses phone cases, screen protectors, and USB cables will have their centroid in the "mobile accessories" region, naturally retrieving related items when we use this centroid as a retrieval query.

Why start here? Because aggregation baselines require zero training, zero GPU compute, and are very straightforward to explain to partner Data Engineering/MLOps teams if you need their help in setting up realtime pipelines. They serve as the floor that any trained model must beat to justify its complexity. This supposedly naïve solution can be surprisingly hard to beat especially when the user sessions are short.

The Five Methods

We implement five aggregation methods, each building on the previous.

Method 1: Simple Mean

The simple mean computes the centroid of all item embeddings in the user's history, giving equal weight to every item. It has no hyperparameters and serves as the absolute simplest baseline. Its primary weakness is that in the absence of any relative weighting between the items (such as through time decay), old interests dilute current intent because a user who browsed laptops last week and phone cases today gets an embedding somewhere between "laptops" and "phone cases," neither fish nor fowl.

```
# Simple mean aggregation
def simple_mean(item_embeddings):
    """Average all item embeddings equally.
    item_embeddings: (N, dim) array of item vectors
    """
    return item_embeddings.mean(axis=0)

# Usage: query the FAISS index with the user embedding
# user_emb = simple_mean(history_embeddings)
# distances, indices = faiss_index.search(user_emb, k=100)
```

Method 2: Last-K Mean

Rather than averaging the entire history, Last-K Mean averages only the K most recent items. This captures current intent without historical dilution. The design choice of K can be either an arbitrary number (e.g., 3 or 5) or grounded in the data. In the worked tutorials, we sweep over the 25th, 50th, and 75th percentile of session or history length distributions, ensuring that K values are tuned to the actual interaction patterns in the dataset. If you want to go a little bit more sophisticated, you can try category-specific percentile length. For example, in a retail setting for products that require extensive research & comparisons (such as a big appliance like washer-dryer), then a longer sequence length is appropriate. Compared to small ticket items like USB charger cables.

Method 3: Exponential Decay

Exponential decay assigns each item a weight that decreases exponentially with its distance from the most recent interaction. The half-life parameter controls how quickly old items lose influence: at a distance equal to the half-life, an item's weight is half that of the most recent item's weight. This provides a smoother alternative to Last-K's hard cutoff—every item contributes, but recent items contribute much more.

The half-life is derived from the data. For short sessions, we can set the half-life relative to the median session length so that the oldest item in a typical session retains moderate weight. For longer histories, we place the 50% weight mark at roughly the midpoint of a typical history, ensuring that the model has a useful gradient across the full interaction sequence.

```
# Exponential decay aggregation
import numpy as np

def exponential_decay(item_embeddings, half_life):
    """Weight items by recency using exponential decay.
    Most recent item is last in the array.
    """
    n = len(item_embeddings)
    positions = np.arange(n)  # 0 = oldest, n-1 = newest
    distances = (n - 1) - positions  # distance from newest
    weights = np.power(0.5, distances / half_life)
    weights /= weights.sum()
    return (weights[:, None] * item_embeddings).sum(axis=0)
```

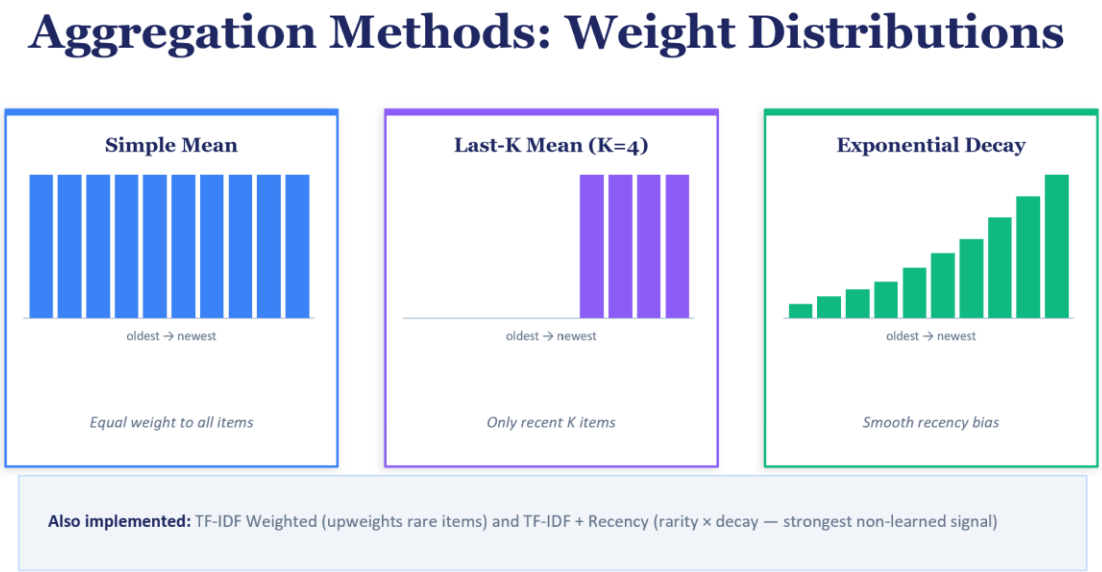


FIGURE 2: Comparison of aggregation method weights over a session of length 10: simple mean (flat line), Last-K (step function), and exponential decay (smooth curve). Shows how each method distributes attention across the history.]

Method 4: TF-IDF Weighted Mean

TF-IDF weighting upweights rare items and downweights popular ones. The intuition is that a user who clicks on a heavily trafficked homepage article should not have their embedding dominated by it: that click tells us nothing distinctive about the user. But a click on a niche article about robotics is a strong signal of genuine interest. The IDF (Inverse Document Frequency) score naturally captures this: items appearing in many sessions have low IDF, while rare items have high IDF.

In our implementation, TF is effectively binary (most items appear once per session), so the weight reduces to pure IDF. The IDF vocabulary is computed on only the training data to prevent test leakage.

To be precise about the implementation: in our TF-IDF weighting, the "term" is the exact item ID (e.g., a product ASIN or an article identifier), not any textual content derived from the item. The "document" is a single user's browsing session (for Amazon KDD) or a single user's full click history (for MIND). The "document frequency" of an item is therefore the number of distinct sessions (or users) in which that item appears. An item that appears in many sessions has high document frequency and receives a low IDF weight; an item that appears in only a few sessions has low document frequency and receives a high IDF weight.

The IDF formula uses additive (Laplace) smoothing: IDF(item) = log((N + 1) / (df(item) + 1)), where N is the total number of sessions (or users) in the training set and df(item) is the count of sessions containing that item. The smoothing constant of 1 prevents division by

zero for items that might not appear in the training set but appear at evaluation time. Crucially, the IDF vocabulary is computed exclusively from the training sessions—the evaluation sessions are not used in the frequency calculation, preventing any form of test leakage. If an evaluation session contains an item not seen during training, that item receives a default IDF weight equal to log((N + 1) / 1), which is the maximum possible weight—treating unseen items as maximally rare.

Because most items appear at most once per session (users rarely click the same product twice in a browsing session), the term frequency (TF) component is effectively binary—either the item is in the session (TF = 1) or it is not. This means the weighting reduces in practice to pure IDF: the weight of each item in the aggregation is determined solely by how commonly that item appears across sessions in the training data, not by how many times the user interacted with it within a single session.

Method 5: TF-IDF + Recency (Combined)

The final aggregation method combines the strengths of Methods 3 and 4 by multiplying the IDF weight (rarity) with the exponential decay weight (recency). A rare item interacted with recently receives the highest weight, with the premise that this is the strongest signal available. This method establishes the ceiling for non-learned aggregation and sets the bar that trained models in later sections must beat.

Evaluation Protocol

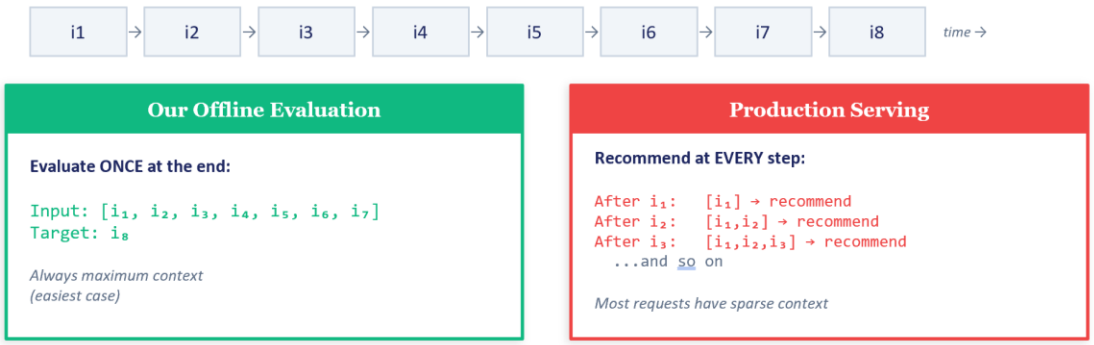
In real life, the evaluation methodology will depend on what proxy metrics we are optimizing for. For the purposes of the companion tutorial, we evaluate using the same next-item retrieval paradigm as we did in Chapter 10. For Amazon KDD, each session provides a sequence of items; we aggregate the history items into a session embedding and attempt to retrieve the actual next item from the full product catalog using FAISS. For MIND, we aggregate a user's reading history into a user embedding and attempt to retrieve articles they subsequently clicked. In both cases, items already in the history are excluded from retrieval results.

```
# FAISS retrieval with user embeddings
import faiss

# Build index from all item embeddings (offline)
dim = item_embeddings.shape[1]  # e.g., 384
index = faiss.IndexFlatIP(dim)  # inner product for cosine sim
faiss.normalize_L2(item_embeddings)  # normalize for cosine
index.add(item_embeddings)

# Query with user embedding (online)
user_emb = aggregate(history_embeddings)  # any method
faiss.normalize_L2(user_emb.reshape(1, -1))
distances, indices = index.search(user_emb.reshape(1, -1), k=100)
```

The key baseline is the last-item only approach from Chapter 10: instead of aggregating the full history, we simply use the embedding of the single last item viewed as the query. This achieved strong results in Chapter 10 and serves as the single strongest comparator.

Evaluation Bias: The Leave-Last-Out Caveat

An important methodological caveat when the evaluation protocol uses leave-last-out: the aggregator always sees the full session history minus the target item. However, in production, a realtime recommendation is generated after every point in the user journey: after 1 item, after 2 items, ….after (N-1) items. Not just at N-1 sequence length. Our evaluation therefore systematically overestimates aggregation quality because it always tests the easiest case (maximum available context).



[FIGURE 3: Diagram illustrating the gap between leave-last-out evaluation (one evaluation point per session at the end) vs. production serving (recommendations needed at every step).]

A more realistic evaluation protocol would randomly truncate sessions at different points, simulating the production distribution of context lengths.

11.3 Sequence Models: SASRec and GRU4Rec

Why Sequence Models?

Aggregation baselines treat all items with fixed weights determined by position, frequency, or simple heuristics. They cannot learn which items in the history are truly predictive of the next action. Consider a session: [headphones, laptop stand, USB hub, mechanical keyboard]. A simple mean gives equal weight to all four items[2]. But "mechanical keyboard" → "keycap set" is a much stronger predictive signal than "headphones" → "keycap set." Sequence models learn these item-to-item predictive patterns from data.

We implement two canonical architectures that represent pervasive baselines in sequential recommendations. SASRec (Self-Attentive Sequential Recommendation) uses a Transformer with causal self-attention, while GRU4Rec uses a Gated Recurrent Unit that processes items

[2] For some use cases where you already know which product category the user wants, you can filter the Recently interacted items to only those within that category. For example in an ecommerce use case for a similar item recommendation within the Product page, you can choose to only have the average embeddings of the subset of user's past interacted items that fall in the same category as the hero item in the Product page.

sequentially. Each brings a different inductive bias: SASRec can attend to any previous item regardless of distance, while GRU4Rec builds up a compressed hidden state that naturally emphasizes recent context.

SASRec and the Self-Attention Mechanism

SASRec applies the Transformer's self-attention mechanism to item sequences. The core idea is that at each position in the sequence, the model computes a weighted combination of all previous items, weights are determined by a combination of item identity and position.. Understanding how this works requires a brief look at the query-key-value attention mechanism.

Each item embedding is projected through three separate linear layers to produce a query vector, a key vector, and a value vector. The query at position t asks "what am I looking for?"; the key at position j answers "what do I contain?"; and the value at position j provides the actual information to pass forward. The attention score between positions t and j is the dot product of the query at t and the key at j, scaled by the square root of the dimension. These scores are normalized via softmax to produce weights that sum to one, and the and the output at position t is the weighted sum of the value vectors at positions 1 through t, with future positions masked out.

Crucially, SASRec applies a causal mask that prevents each position from attending to future items. This ensures that the prediction at position t uses only items at positions 1 through t, matching the production scenario where we must predict the next item from a partial history. Without the causal mask, the model would "cheat" by looking at future items during training, producing embeddings that are useless at inference time.

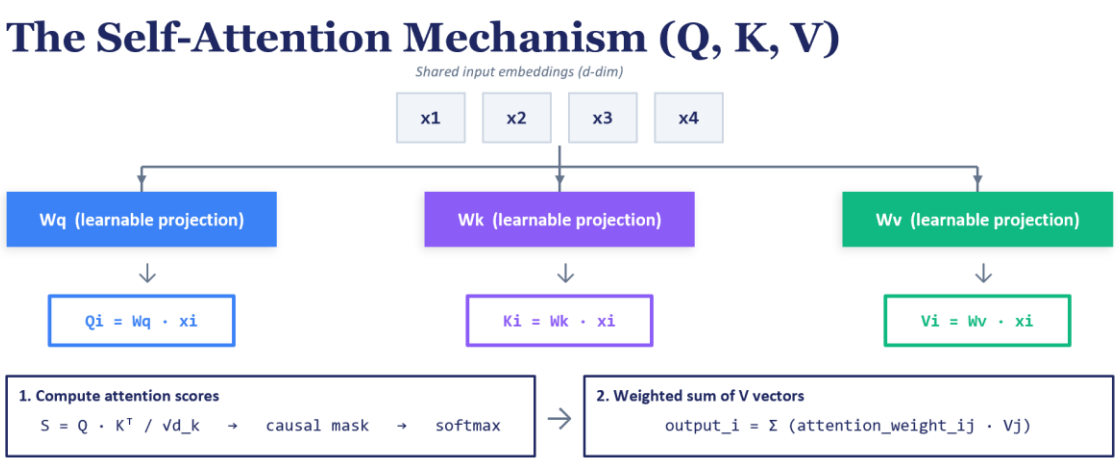


FIGURE 4: Self-attention mechanism diagram showing query, key, and value projections, the attention score computation, causal masking, and the weighted value summation for a sequence of four items.

GRU4Rec and the Gating Mechanism

GRU4Rec processes items one at a time, maintaining a hidden state that evolves with each new item. The Gated Recurrent Unit at its core uses two learned gates: the update gate and the reset gate, to control how information flows through the sequence.

The reset gate determines how much of the previous hidden state to forget when computing a candidate new state. When the reset gate is near zero, the GRU effectively ignores the accumulated history and starts fresh from the current item—useful when a session shifts topics abruptly. The update gate controls how much of the candidate new state to blend with the previous hidden state. When the update gate is near one, the hidden state passes through unchanged, preserving long-range context; when near zero, it is fully replaced by the new candidate. Together, these gates let the GRU learn when to remember and when to forget, without requiring the designer to hand-code recency weights as in the aggregation baselines.

Compared to SASRec's attention mechanism, the GRU compresses the entire history into a fixed-size hidden state, which means it cannot directly attend to a specific early item the way attention can. For very long sequences, this compression can lose information. However, for short sessions the compressed state is often sufficient, and the GRU's sequential inductive bias aligns well with session data where the order of clicks carries strong signal.

Architecture

Normally, the input for both train and serve is a sequence of Item IDs without any metadata. However, its also valid pattern to have the input be a set of item embeddings corresponding to the item IDs that the user interacted with. This is useful in mitigating cold start issues for new items or those from the long tail where the interactions are sparse. A new item/long tail item has no/little interaction history. A pre-trained embedding from content features gives the sequential model something meaningful immediately. At serving time, you'd look up the pre-trained embedding for each item ID the user interacted with and feed that sequence into the SASRec/GRU model.

Both sequential models follow the same high-level pipeline. The input is a sequence of frozen item embeddings from the prior chapter (we chose the 384-dimensional SBERT vectors). A linear projection compresses these to the model's working dimension (128). The sequence model processes the compressed sequence and produces a 128-dimensional output at each position. A final linear projection maps back to 384 dimensions which is in the same space as the frozen item embeddings: enabling direct retrieval using FAISS.

FIGURE 5: Architecture diagram showing the input projection (384→128), sequence model (SASRec or GRU4Rec), and output projection (128→384) with L2 normalization.

The output projection deserves special attention. It maps from the model's 128-dimensional internal space back to the 384-dimensional item embedding space. But same dimensions do not automatically mean the same semantic space. What forces alignment is the training objective, not the architecture. The MNR loss explicitly pushes the model's output vectors toward the correct target's SBERT embedding and away from all other items' embeddings. Over training, the output projection learns to "speak the language" of the frozen SBERT embedding space.

Both models are remarkably small: in the worked examples, SASRec has approximately 500K parameters and GRU4Rec approximately 300K. These are trivial compared to the frozen item embedding matrix they operate over, which can be 10s of millions of megabytes. The models train in minutes on GPU for datasets of this scale.

Training: MNR Loss with Multi-Position Training

The training uses the same MNR (Multiple Negatives Ranking) loss from the prior chapter's contrastive fine-tuning, maintaining narrative continuity and allowing students to see the same concept applied in a new context.

A key training innovation is multi-position training. Rather than training only on the final prediction (given the full session, predict the next item), we extract a training signal at every position in the sequence. A session [i_1, i_2, i_3, i_4] with next_item i_5 produces multiple training signals: after i_1 predict i_2, after [i_1, i_2] predict i_3, and so on. This dramatically increases the effective training data for short sessions and teaches the model to produce good embeddings from partial sequences. Thereby matching the production scenario where recommendations must be generated at every step.

```
# MNR loss with multi-position training (SASRec)
def train_step(model, batch_sequences, batch_targets):
    """Each sequence yields predictions at every position.
    batch_sequences: (B, max_len, dim) padded item embeddings
    batch_targets: (B, max_len, dim) shifted targets
    """
    outputs = model(batch_sequences)  # (B, max_len, dim)
    # Gather valid (non-padded) predictions and targets
    preds = outputs[mask]    # (N_valid, dim)
    targets = batch_targets[mask]  # (N_valid, dim)
    # MNR: each pred should be closest to its own target
    preds = F.normalize(preds, dim=-1)
    targets = F.normalize(targets, dim=-1)
    logits = preds @ targets.T  # (N_valid, N_valid)
    labels = torch.arange(len(preds), device=logits.device)
    loss = F.cross_entropy(logits / temperature, labels)
    return loss
```

Key Takeaway: When Do Sequence Models Help?

Sequence models are not universally better than simple aggregations. Their advantage depends on the quality of the training signal (MNR loss needs distinguishable targets), the density of the embedding space (dense topic clusters confuse contrastive learning), and the match between training and evaluation. In production systems with longer sessions, more interaction data, and carefully constructed negative samples, sequence models typically show larger gains. Our short-session e-commerce dataset and topically clustered news dataset represent particularly challenging scenarios for these models.

Learnable Item ID Embeddings: An Important Negative Result

A natural question is whether replacing frozen SBERT embeddings with learnable item ID embeddings (a standard nn.Embedding lookup table trained jointly with the sequence model) would improve performance. The exercises demonstrate that this significantly underperforms frozen SBERT on both datasets.

The reason is data sparsity. The Amazon KDD catalog contains hundreds of thousands of products but provides only a modest number of interactions per product on average, far too sparse for a high-dimensional embedding to learn meaningful patterns from scratch. Frozen SBERT provides strong content-based priors that generalize to items with few interactions. Learnable embeddings require sufficient interaction density per item—a threshold that this dataset does not meet. This negative result sets the stage for Section 11.4, where LightGCN's graph structure provides a richer training signal that makes learnable embeddings viable.

11.4 Graph Neural Networks: LightGCN

Why LightGCN?

LightGCN (He et al., SIGIR 2020) addresses this through two key differences: it learns its own embeddings from scratch (avoiding the dense cluster problem), and it leverages the collective behavior of all users through graph propagation (providing richer training signal than sequential prediction alone).

LightGCN was chosen for this section for the following reasons. It is one of the simplest possible GNN architecture: the entire model is just sparse matrix multiplication and layer averaging, with no weight matrices, activation functions, or dropout. This makes it ideal as a textbook introduction to graph-based recommendations (even though this architecture came a few years after other popular GNNs). It is also the most cited GNN-for-RecSys paper, making it essential knowledge for practitioners in this domain.

Author's Note: LightGCN requires persistent user IDs to construct the user–item graph. Amazon KDD has no user IDs (each row is an anonymous session), so LightGCN is applied only to the MIND dataset in this chapter.

Bipartite Graphs for Recommendation

Before introducing bipartite graphs specifically, it is helpful to establish what a graph is in general terms. A graph is a mathematical structure consisting of nodes (also called vertices) and edges (also called links) that connect pairs of nodes. Nodes represent entities—in our context, users and items—and edges represent relationships between them, such as "user U clicked on item I." A graph can be directed (edges have a direction, like "user U purchased item I") or undirected (the relationship is symmetric). Graphs are a natural way to represent the web of interactions between users and items in a recommendation system, and graph neural networks learn to extract patterns from this structure.

For readers new to graph neural networks, we begin with a conceptual primer. A bipartite graph has two types of nodes. In the case of recommendations, it can be users and items. In the case of the Bipartite graph, the graph is constructed such that the edges occur only between different node types (never user-to-user or item-to-item). This is different from say a social media graph where there can be a graph between the user to another user.

In its simplest representation, each edge represents an interaction. For the MIND dataset, since the items are articles, the presence of an edge represents that the user U clicked article I. The graph is static and unordered: the metadata on **when** the interactions happened and in **what order** are not captured explicitly in the edge information. The graph only knows "who clicked what," not "when" or "in what sequence." This is a fundamental difference from Section 11.3's sequence models, which preserve temporal ordering. Note that this is not the only way to represent the interaction history, but is the simplest one for pedagogical purposes.

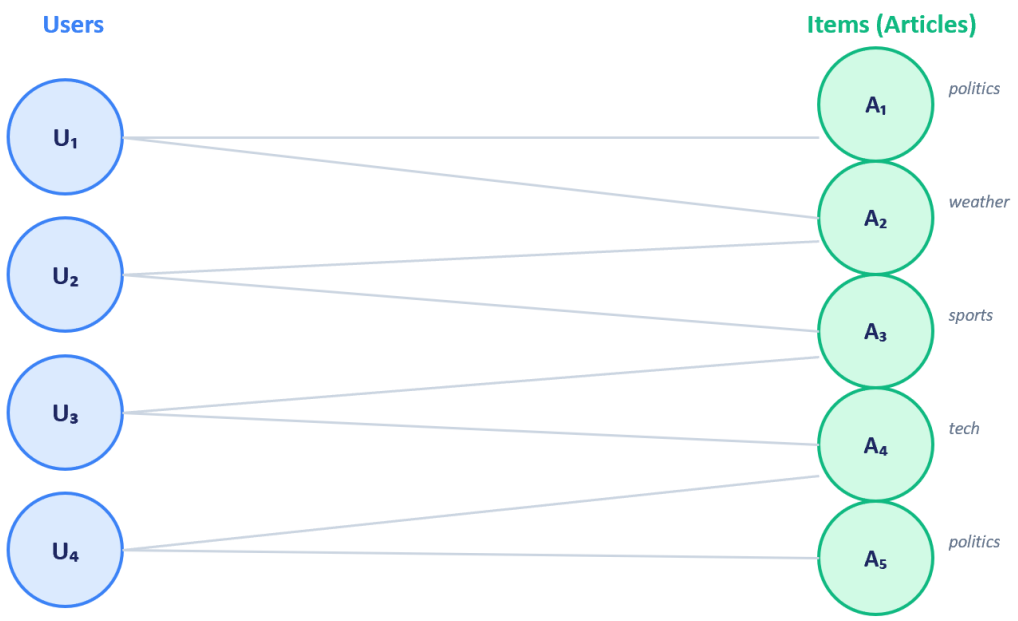
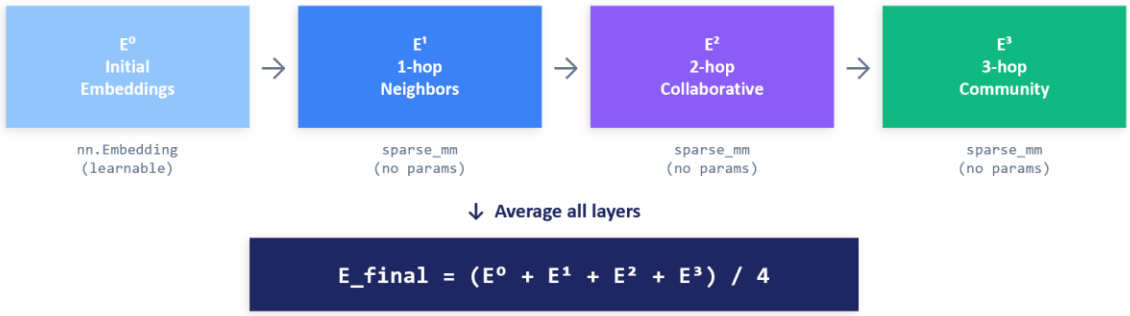


FIGURE 6: Bipartite graph diagram showing users on the left, articles on the right, with edges connecting users to articles they clicked. Illustrate 1-hop, 2-hop, and 3-hop neighborhoods.

How LightGCN's Graph Propagation Works

LightGCN starts by assigning each user and each item a learnable embedding vector (initialized randomly). Then it performs K rounds of neighborhood aggregation via sparse matrix multiplication. In each round, every node's embedding becomes the weighted average of its neighbors' embeddings. At the first layer (1-hop), each user absorbs information from the articles they read, and each article absorbs information from the users who read it: this is a typical implementation of direct neighbor aggregation. At the second layer (2-hop), each user now indirectly receives information from other users who read the same articles, capturing the classic collaborative filtering signal of "users who read X also read Y." At the third layer (3-hop), information propagates further through chains of shared articles, capturing broader community-level patterns.



[FIGURE 7: LightGCN message passing visualization showing how information flows through 1-hop, 2-hop, and 3-hop neighborhoods in a bipartite user-item graph, with arrows indicating the direction of embedding propagation at each layer.]

The adjacency matrix is symmetrically normalized so that high-degree nodes do not dominate. A user who reads many articles sends a weaker signal per edge than a user who reads few; a popular article clicked by many users contributes less per edge than a niche article clicked by a handful.

After K rounds of propagation, LightGCN averages the embeddings from all layers (including the initial layer 0) to produce the final representation. Each layer captures a different scale of information: layer 0 is the node's own identity, layer 1 captures direct neighbors, layer 2 captures 2-hop collaborative patterns, and so on. The averaging combines local identity with increasingly broad collaborative signal.

The Simplicity of LightGCN

A critical point that distinguishes LightGCN from standard neural networks: the only learnable parameters are two embedding tables (one for users, one for items). The graph propagation layers have zero learnable parameters because the authors chose to remove the weight matrices and activations that existed in their predecessor (NGCF). They perform pure matrix multiplication with the fixed normalized adjacency matrix, with no per-layer weight matrices or activation functions.

This is counterintuitive. In standard neural networks, each layer has its own weight matrix that transforms the data. In LightGCN, the graph layers are like mirrors in a hall of mirrors: they reflect and propagate information, but the mirrors themselves have no adjustable knobs. The only thing you can adjust is what you put into the mirrors (the initial embeddings). LightGCN paper's key finding was that removing all per-layer parameters actually improves recommendation quality on standard benchmarks. This is because the very nature of the graph structure already encodes sufficient information, and per-layer transforms add parameters that overfit without adding useful expressiveness.

```
# LightGCN forward pass (simplified)
def lightgcn_forward(user_emb, item_emb, adj_norm, n_layers=3):
    """Propagate through the bipartite graph.
    adj_norm: symmetrically normalized adjacency (sparse)
    """
    all_emb = torch.cat([user_emb, item_emb], dim=0)
    layer_embs = [all_emb]  # layer 0 = raw embeddings
    for _ in range(n_layers):
        all_emb = torch.sparse.mm(adj_norm, all_emb)
        layer_embs.append(all_emb)
    # Average across all layers (0 through K)
    final_emb = torch.stack(layer_embs, dim=0).mean(dim=0)
    n_users = user_emb.shape[0]
    return final_emb[:n_users], final_emb[n_users:]
```

More complex GNN architectures do add learnable parameters per layer. NGCF (Wang et al., SIGIR 2019) adds weight matrices; GAT adds learnable attention weights per edge; GraphSAGE uses learnable aggregation functions. These are worth exploring after understanding LightGCN, but the simplicity of LightGCN makes it the right starting point.

Loss Functions: BPR vs. MNR

We train LightGCN with two loss functions to compare their behavior. BPR (Bayesian Personalized Ranking) is a pairwise loss that compares one positive item against one randomly sampled negative item per training example. It pushes the positive item's score above the negative item's score by optimizing the log-sigmoid of their score difference. Formally, for a user u with positive item i and negative item j, BPR minimizes $-\log \sigma(\text{score}(u,i) - \text{score}(u,j))$, where σ is the sigmoid function. This pairwise formulation is more stable than pointwise losses because it does not require calibrated absolute scores. It only needs to get the relative ordering right. The gradient signal is weak (only one comparison per update), but training proceeds steadily.

MNR (Multiple Negatives Ranking) uses in-batch negatives, the same approach from Sections 10.5 and 11.3. With a large batch size, each positive gets many negative comparisons. This provides richer gradient signal but can fail when targets are indistinguishable.

Instead of random initialization, another implementation is by projecting a pre-trained embeddings to LightGCN's working dimension using PCA and use these projected vectors as

the initial item embeddings. In the worked tutorials, the prior chapter's SBERT embeddings is leveraged.

Theoretically, MNR stands to gain far more from SBERT initialization because it processes many negatives per update and can immediately exploit the meaningful topic structure provided by SBERT. With random initialization, MNR must first learn coarse topic boundaries before making fine-grained distinctions. SBERT initialization effectively skips this phase. BPR, processing only one negative at a time, makes slow incremental adjustments regardless of starting position, so the initial geometry matters less.

This approach can be thought of as a "poor man's PinSage." Full PinSage (Ying et al., 2018) incorporates content features through learned message-passing aggregators at every layer and this approach requires graph sampling, feature transformation matrices, and significantly more complexity. The alternative approach achieves a simpler version of the same idea: inject content knowledge once at initialization, then let LightGCN's parameter-free graph propagation refine it.

Cold-Start User Handling

A practical consideration for any graph-based method is handling users not present in the training graph. Our evaluation users are deliberately excluded from training (cold-start evaluation). For these users, we compute their embedding by averaging the post-propagation LightGCN embeddings of items in their history. This is equivalent to one round of LightGCN propagation for a new user node. This is the same pattern that would be used in production: known users get pre-computed embeddings from the graph; new users get on-the-fly embeddings from averaging item vectors.

```
# Cold-start user embedding from item history
def cold_start_user_embedding(history_item_ids, item_embeddings):
    """Compute embedding for a user not in the training graph.
    item_embeddings: post-propagation LightGCN item vectors
    """
    history_embs = item_embeddings[history_item_ids]
    user_emb = history_embs.mean(dim=0)  # simple average
    return F.normalize(user_emb, dim=0)
```

Production Inference Pattern

In production, the inference pattern splits into offline and online components. The offline step (run periodically, for example hourly or daily) executes the full graph forward pass to produce all user and item embeddings, then indexes the item embeddings in a vector database. The online step (per-request, real-time) looks up cached embeddings for known users and queries the vector database; for cold-start users, it averages the cached item embeddings of their recent interactions and queries the database. This offline/online split is standard for graph-based recommendation systems and ensures sub-millisecond serving latency.

11.5 Reusable Embeddings in Industry

The techniques explored in Sections 11.2–11.4 are building blocks—they demonstrate the fundamental approaches on accessible datasets. Production systems at scale combine these ideas in architectures that emphasize reusability across multiple recommendation surfaces.

Multi-Interest User Representations

Alibaba's MIND[3] model (Multi-Interest Network with Dynamic Routing) addresses a limitation of single-vector user embeddings: one vector may be insufficient to capture a user's diverse interests. A user who shops for both electronics and baby products has interests in two distinct regions of the embedding space, and a single centroid between them represents neither accurately. MIND produces multiple user embedding vectors (typically three to five), each representing a distinct interest cluster. At retrieval time, each interest vector is used as a separate query, and the results are merged. This increases the diversity of retrieved candidates and ensures that the system covers all of the user's interests, not just the dominant one.

Author's Note: Alibaba's MIND model should not be confused with the Microsoft MIND news dataset used in our exercises. They share the same acronym but are entirely different.

Binary Embedding Compression

At very large scale (tens of billions of items), even compact float embeddings become expensive to store and query. Tencent's BEBR (Binary Embedding-Based Retrieval) system compresses real-valued embeddings into binary codes (bit vectors), reducing storage dramatically and enabling fast similarity computation using CPU SIMD bit operations. The binarization model is trained to preserve retrieval accuracy with negligible loss. This illustrates an important production concern that our tutorial-scale datasets do not surface: at web scale, the efficiency of the embedding representation itself becomes a first-order design constraint.

Author's Note: Investing in reusable embeddings is a high-ROI strategic ML priority because of second-order benefits beyond accuracy improvement. In the author's experience, developing item embeddings forces product and business teams to critically examine catalog metadata quality, clean taxonomy, and land new data sources. Similarly, thinking about how user embeddings will be leveraged highlights data ingestion latencies that can only be solved through streaming pipelines, unblocking other teams' low-latency use cases.

11.6 Connecting the Paradigms: When to Use What

The three paradigms explored in this chapter (aggregation, sequential, and graph-based) are not mutually exclusive. In practice, production systems often combine them. The following decision framework can guide practitioners.

For cold-start scenarios with no user IDs or very short sessions, start with aggregation baselines, specifically the last-item approach or exponential decay. These require no

[3] Not to be confused with the MIND Microsoft news dataset

training and are hard to beat for short, intent-driven sessions. They are the right starting point for any new recommendation system.

When moderate interaction data is available and users are identified, train sequence models (SASRec or GRU4Rec) on interaction histories. These learn which items in the history are most predictive, going beyond fixed-weight aggregation. Ensure your targets are sufficiently distinguishable for contrastive training to work.

When a rich interaction graph exists with persistent user IDs, add graph-based methods (LightGCN or more advanced GNN variants) to capture collaborative patterns across users. Initialize item embeddings with content features for the best of both worlds. Graph methods especially shine when individual user histories are sparse but the collective graph is dense.

For production deployment at scale, combine approaches: use aggregation for cold-start users, pre-trained sequence models for users with short histories, and graph-based embeddings for users with rich interaction data. Weight the combination by the confidence of each signal—more history means more weight on learned embeddings.

11.7 Summary

This chapter explored three paradigms for constructing user embeddings from item interaction histories, with several findings that challenge conventional wisdom.

For short, intent-driven sessions (e-commerce), the single last item viewed is the strongest predictor of the next action. Aggregating in additional context actually dilutes this signal. This is the most practically important finding for engineers building session-based recommendations.

For longer user histories (news, media), aggregation adds clear value over single-item baselines. Recency is the strongest single signal, and combining recency with rarity (TF-IDF + decay) provides the best non-learned approach.

Sequence models (SASRec, GRU4Rec) beat most aggregation methods but fail catastrophically when combined with frozen content embeddings in topically clustered catalogs. The MNR mode collapse finding is a cautionary tale about the interaction between loss functions and embedding space geometry.

Graph neural networks (LightGCN) with learnable embeddings resolve the MNR mode collapse, achieving the best results in the chapter. Initializing with content embeddings from Chapter 10 further improves performance substantially, demonstrating that content and collaborative signals are complementary.

User embedding quality is bounded by item embedding quality. The same aggregation method applied to better item embeddings consistently outperforms the same method applied to worse ones. Invest in item embeddings first, user aggregation sophistication second.

Together with the prior chapter, this chapter completes the representation layer of a recommendation system. Items are embedded in a space that captures both content and

behavioral similarity; users are embedded in the same (or a compatible) space based on their interaction histories. With both representations in hand, candidate retrieval reduces to a nearest-neighbor search, and the retrieved candidates can be passed to the ranking models of Chapters 6–8 for final scoring.

Exercises

Exercise 1: Random-Truncation Evaluation

Implement a random-truncation evaluation protocol for the aggregation baselines. Instead of always using the full session history, randomly truncate sessions at different points to simulate the production scenario. Compare results with the leave-last-out evaluation.

Exercise 2: Embedding Quality Comparison

Using simple mean aggregation, compare user embeddings built from fine-tuned SBERT, zero-shot SBERT, and Item2Vec embeddings on Amazon KDD. What does this tell you about the relative importance of item embedding quality vs. aggregation sophistication?

Exercise 3: Attention Visualization

Extract and plot SASRec attention weights for example users on MIND. For each user, show which history articles receive the highest attention when predicting different targets.

Exercise 4: LightGCN Layer Analysis

Compute retrieval metrics using each individual LightGCN layer's embeddings rather than the averaged final embeddings. Which layer performs best? Does averaging outperform any individual layer?

Exercise 5: Integrating User Embeddings into Ranking

Use the aggregation methods from Section 11.2 to create user embeddings from the SIGIR e-commerce dataset's item vectors (Chapter 7), then add these as features to the ranking model. Measure the lift in ranking quality from including user embeddings.

References

1. He, X., Deng, K., Wang, X., et al. (2020). LightGCN: Simplifying and Powering Graph Convolution Network for Recommendation. SIGIR.
2. Kang, W.-C. and McAuley, J. (2018). Self-Attentive Sequential Recommendation (SASRec). ICDM.
3. Hidasi, B., Karatzoglou, A., Baltrunas, L., and Tikk, D. (2016). Session-based Recommendations with Recurrent Neural Networks (GRU4Rec). ICLR.
4. Reimers, N. and Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP.
5. Rendle, S., Freudenthaler, C., Gantner, Z., and Schmidt-Thieme, L. (2009). BPR: Bayesian Personalized Ranking from Implicit Feedback. UAI.
6. Henderson, M., Al-Rfou, R., Strope, B., et al. (2017). Efficient Natural Language Response Suggestion for Smart Reply. arXiv:1705.00652.

7. Li, C., Liu, Z., Wu, M., et al. (2019). Multi-Interest Network with Dynamic Routing for Recommendation at Tmall (MIND model). CIKM.
8. Zhai, J., Liao, L., Liu, X., and Wang, Y. (2024). Actions Speak Louder Than Words: Trillion-Parameter Sequential Transducers for Generative Recommendations (Netflix Foundation Model). ICML.
9. Ying, R., He, R., Chen, K., et al. (2018). Graph Convolutional Neural Networks for Web-Scale Recommender Systems (PinSage). KDD.
10. Wang, X., He, X., Wang, M., et al. (2019). Neural Graph Collaborative Filtering (NGCF). SIGIR.
11. Hamilton, W., Ying, Z., and Leskovec, J. (2017). Inductive Representation Learning on Large Graphs (GraphSAGE). NeurIPS.
12. Zhang, Y., Liu, Y., Han, P., et al. (2023). Binary Embedding-based Retrieval at Tencent (BEBR). KDD.
13. Haldar, M., Abdool, M., Ramanathan, P., et al. (2020). Applying Deep Learning To Airbnb Search. KDD.
14. Wu, F., Qiao, Y., Chen, J.-H., et al. (2020). MIND: A Large-scale Dataset for News Recommendation. ACL.
15. Barkan, O. and Koenigstein, N. (2016). Item2Vec: Neural Item Embedding for Collaborative Filtering. IEEE MLSP.